Research Pipeline Design

2026-10-22

Overview

Commands for working with the literature in Claude Code: find the papers that matter on a topic, take a single paper apart, download a paper's PDF with a clean file name, and make the figures and results tables that go into your own paper.

The problem both solve is trust. A literature answer is only useful if every paper in it is real, correctly attributed and worth reading, and if the analysis separates what the paper says from what the reader should think of it. So the pipeline is built around **trusted sources** and **verification before citation**.

Command	Input	Output
`/research-list-paper`	a topic or keywords	`.research/lists/<date>-<topic>.md` — the key papers, foundational and recent
`/research-summary-paper`	a title, arXiv ID, DOI or PDF	`.research/papers/<author><year>-<title>.md` — big picture, claims, evidence, limitations
`/research-download-paper`	a title, link, arXiv ID, DOI, or numbers from the last list	`.research/pdfs/<year>. <title> (<venue>).pdf` + `.txt` metadata, `index.txt`, `references.bib` (or a folder the user names)
`/research-figure`	a description of what to show	`.research/figures/<date>-<slug>/figure.{json,pptx,pdf,png}`
`/research-table`	data to organize — pasted, a CSV, or numbers from named papers	`.research/tables/<date>-<slug>/table.{json,pptx,pdf,png}`
`/research-animation`	a description of what to animate	`.research/animations/<date>-<slug>/{scene.py, <Scene>.mp4, <Scene>.gif, steps.png}`

Principles

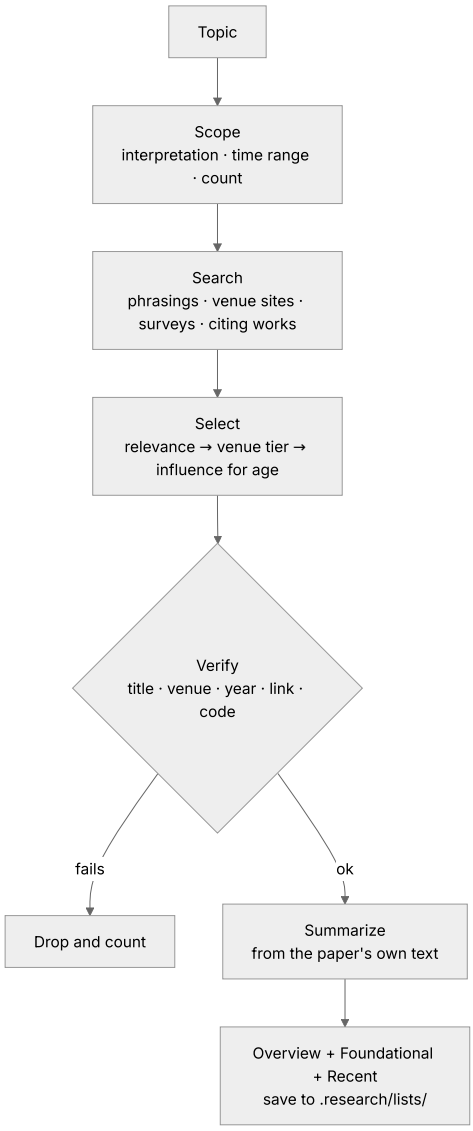
Nothing unverified gets cited. Every title, venue, year, link and code repo is confirmed from a fetched page before it appears. Search snippets and memory are for finding candidates, never for citing them. An entry that can't be verified is dropped, and the output says how many were dropped.

Trusted venues first. Peer-reviewed papers from established venues (CVPR, ICCV/ECCV, SIGGRAPH, NeurIPS, ICML, ICLR, IEEE journals, ...) rank first. An arXiv-only paper is included only with a stated reason: adoption, the authors' track record, or a very recent result that already sets a direction.

The paper's voice and ours stay apart. Claims are attributed to the authors, with section and table references and exact numbers. Criticism is labeled as assessment and must point at something specific.

Output in the user's language. The kit is English; summaries and analyses follow the language of the request. Names of papers, venues, methods and datasets stay as published.

/research-list-paper



Each entry is a table row: a slide-ready citation (`Kerbl et al., <Title> (SIGGRAPH 2023)`), links to the paper, the official code (or `–`) and a page that serves its BibTeX, why the paper is key, and a 2–3 sentence summary of the problem it tackles and how — or what it claims. An overview paragraph on top traces how the topic developed.

Defaults: 10–15 papers, split between foundational work and the last ~2 years. The request can change the range, the count or the focus.

/research-download-paper

Resolves each paper to its published title, venue and year (same checks as a list entry), then saves the PDF — to `.research/pdfs/` by default, or to the folder the user names in the request (their Downloads folder included) — as `<year>. <title> (<venue>).pdf` — e.g. `2024. SplaTAM - Splat, Track & Map 3D Gaussians for Dense RGB-D SLAM (CVPR).pdf`. The venue's camera-ready PDF comes first, arXiv is the fallback; only open, legitimate copies are used. A paywalled paper with no open copy is reported with its link instead.

Beside each PDF goes a same-named `.txt` so the paper is easy to look up and cite later: authors, venue, the citation (slide-ready `Kerbl et al., <Title> (SIGGRAPH 2023)`, short and full), the links (venue page, DOI, arXiv, verified code and project page, the PDF's source), the BibTeX with where it came from (venue page → Crossref → arXiv) and the abstract. The per-paper files are the record; from them the script regenerates `index.txt` (one line per paper) and `references.bib` (all entries, for LaTeX) in the same folder on every download — one file per paper keeps a PDF and its metadata together when moved or deleted, and the two generated files give the overview without ever drifting from it.

/research-animation

A short animation for a talk or project page — 10–30 s, one message, 4–8 steps — written as a Manim scene (`scene.py` is the spec) with the `paperanim` helpers, which give it the figures' palette, a white background and one sans font, so animation and figures read as one set. Scenes can be 2D (pipelines, data flow) or 3D (`ThreeDScene` — back-projection, camera rays, point clouds, with a moving camera and labels fixed to the frame).

- **Review without watching.** Claude can't watch video, so `animtool.py render` turns a 480p draft into one image, `steps.png` : the last frame of every `play()` , with steps that change nothing (a `wait`) dropped. One image per round keeps the fix loop cheap; Manim's cache re-renders only the changed steps. Errors come back as one line (message + `scene.py` line); the full log stays in a file. Only the last round renders 1080p30 MP4 plus a GIF for slides.
- **Dependencies stay out of the user's Python.** Manim needs cairo, pango and ffmpeg, which pip can't install on Linux without system packages. `animtool.py setup` — run once per machine after Claude asks — downloads micromamba if needed and installs Manim from conda-forge into `~/.cache/research-pipeline/manim/` (Windows `%LOCALAPPDATA%\research-pipeline\manim\`): the same path on Windows, macOS and Linux, no admin rights, shared by all projects. `install.sh --with-animation` / `install.ps1 -WithAnimation` does it at install time instead. LaTeX is optional; without it, formulas render through matplotlib or Pango markup.

/research-figure and /research-table

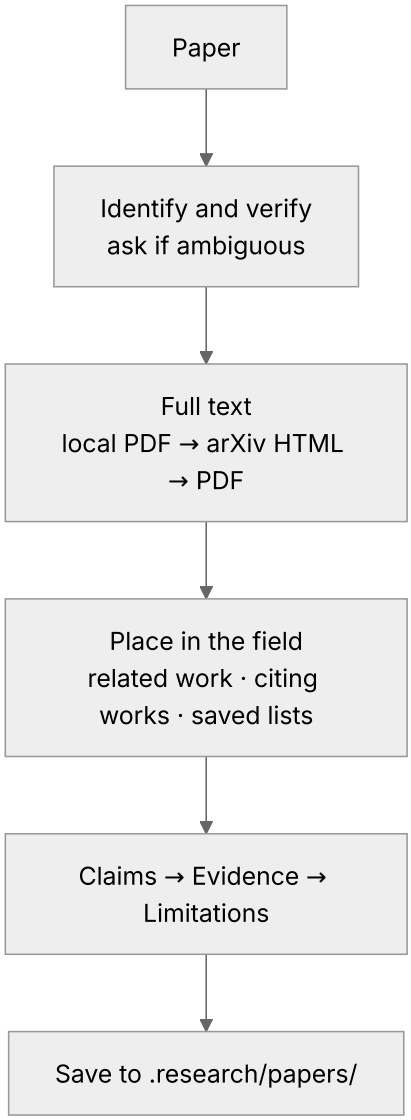
Both write a JSON spec and build it with `figuretool.py` (skill `research-figures`) into an **editable PPTX** — real PowerPoint shapes and tables, not a picture — plus a PDF for `\includegraphics` and a PNG preview. The build crops the canvas to the ink with minimal margins on all four sides and warns about

overlapping shapes, colliding text, arrows crossing unrelated elements, elbow arrows whose arrowhead segment is too short to show the turn, and text that won't fit (measured with Arial glyph widths; text never breaks a word and only breaks lines where the spec says). Sub-figure labels `(a)`, `(b)` are placed by the script, centered under each panel on a shared baseline; Claude fixes every warning, then reads the preview for anything else before reporting (up to 3 rounds). Intended overlaps are marked explicitly. The spec is kept, so later changes are small edits and a rebuild.

- **Figures** are sized for print (3.25 in single column, 6.875 in full width) with real font sizes (≥ 7 pt), a small colorblind-safe palette, and no decoration. Pipelines, concept comparisons, architectures, or data plots (matplotlib, placed as an image).
- **Tables** follow booktabs: heavy top and bottom rules, a light rule under the header, trimmed rules under column groups, no vertical lines, best **bold** and second **underlined** per column, `↑`/`↓` in headers, and footnotes (`*`, `†`, `‡`) on specific rows explained under the table.
- Numbers in either come only from the user or from the paper's own text, with the source noted.

PDF export uses PowerPoint (Windows COM) or LibreOffice; without them the PPTX is still produced.

/research-summary-paper



Section	Content
Big picture	the line of work it extends, the gap it targets, what came after (up to 10 follow-ups, grouped by direction)
Method in brief	how it works: inputs, core idea, outputs
Claims	what the authors claim, numbered, main contribution first; compound claims split
Evidence	per claim: assumptions, experiments (setup, numbers, ablations), proofs or explanations if any, and a one-line verdict
Limitations	acknowledged by the authors · gaps in the argument · room for improvement
Bottom line	what to take away and how much to trust it

Two checks are built in because they find the real issues most often: each claim is compared with the conditions it was actually tested under, and headline numbers are checked against the detailed tables.

If the full text or a linked supplement isn't available, the analysis says so at the top and stays within what was actually read. The chat shows only the bottom line and the verdicts; the full analysis is in the file.

Verification levels

Level	Applies to	Requires
Full check	each paper in a list; the paper being analyzed	title from the paper's page, venue + year from the venue's own record or DOI, a working link, official code confirmed
Light check	papers mentioned in passing (e.g. follow-ups)	one metadata record (OpenAlex, Crossref, Semantic Scholar or arXiv)

Anything that rests on a weaker source or couldn't be reached is written in the output's source note rather than silently asserted.

Sources

Lookups run through `researchtool.py` in the skill's `scripts/` folder — search, metadata, citing works, venue pages, GitHub, BibTeX links and full text — so each run prints only what's needed instead of raw pages.

The skill `research-sources` holds the shared rules: venue tiers, arXiv-only criteria, the verification table, link priority and lookup services. Every command reads it first.

Discovery uses web search. Verification uses arXiv, Crossref, OpenAlex and Semantic Scholar APIs plus the venue sites (CVF Open Access, OpenReview, PMLR, NeurIPS proceedings, ACM DL). All work without keys; if `SEMANTIC_SCHOLAR_API_KEY` or `OPENALEX_API_KEY` is set, it is used. The pipeline never signs up for keys or sends the user's details anywhere.

Files

```
<project>/
├── CLAUDE.md                          research block (marker) — commands, language,
integrity
├── .claude/
│   ├── commands/
│   │   ├── research-list-paper.md
│   │   ├── research-summary-paper.md
│   │   ├── research-download-paper.md
│   │   ├── research-figure.md
│   │   ├── research-table.md
│   │   └── research-animation.md
│   └── skills/
│       ├── research-sources/          source & verification rules + scripts/researchtool.py
│       ├── research-figures/          figure & table rules + scripts/figuretool.py
│       └── research-animations/       animation rules + scripts/animtool.py, paperanim.py
└── .research/
    ├── lists/                         /research-list-paper output
    ├── papers/                        /research-summary-paper output
    ├── pdfs/                          /research-download-paper output (gitignored)
    ├── figures/                       /research-figure output
    ├── tables/                        /research-table output
    └── animations/                    /research-animation output (render cache .media/
gitignored)
```

`/research-summary-paper` reads `.research/lists/` when placing a paper in its field, so earlier lists make later analyses better informed.